

ANS w Elblągu
Instytut Informatyki Stosowanej
Im. Krzysztofa Brzeskiego

Przetwarzanie równoległe i rozproszone
Studium stacjonarne, sem 4, 2025/2026

Projekt Semestralny

Autorzy:

- Adrian Czarnota 21442
- Dawid Rzepka 21299

Grupa: 2

Data rozpoczęcia: 21.05.2026

Data oddania: 09.06.2026

Repozytorium GitHub: <https://github.com/rzepeeeek/Web-Scraper-Project/>

Cel projektu

Celem projektu było stworzenie aplikacji do pobierania, przetwarzania oraz przechowywania danych (np. Adresy email, linki, numery telefonów, adresy) w architekturze rozproszonej. Użytkownik, korzystając z interfejsu webowego, wprowadza adresy URL, które następnie są przetwarzane przez aplikację. System pobiera dane w sposób asynchroniczny, analizuje je i zapisuje w bazie danych MongoDB.

Projekt miał na celu praktyczne wykorzystanie technologii backendowych oraz konteneryzacji. Kluczowym założeniem było również rozdzielenie aplikacji na niezależne moduły działające jako osobne kontenery Docker, takie jak interfejs użytkownika, silnik przetwarzający oraz baza danych.

Opis działania

Użytkownik wprowadza adresy URL poprzez interfejs webowy, po czym dane są przesyłane do silnika scrapującego. Silnik pobiera zawartość stron internetowych w sposób asynchroniczny (asyncio), a następnie przetwarza dane równoległe przy użyciu multiprocessing. Zebrane dane są analizowane (parsowane) przy użyciu biblioteki BeautifulSoup, a następnie zapisywane w bazie danych MongoDB.

Podczas analizy wyodrębnione są m.in. tytuły stron, adresy e-mail, numery telefonów, odnośniki, nagłówki H1/H2 oraz opisy meta.

Aplikacja działa w architekturze rozproszonej, gdzie poszczególne komponenty komunikują się ze sobą poprzez interfejs sieciowy.

Struktura systemu

System składa się z trzech głównych modułów:

- Interface (Flask + HTML/CSS) - odpowiada za przyjmowanie adresów URL oraz komunikację z użytkownikiem.
- Engine (Silnik scrapujący) - realizuje pobieranie, przetwarzanie i analizę danych.
- Database (MongoDB) - przechowuje dane pozyskane ze stron.

Wszystko uruchamianie jest jako kontenery za pomocą Docker Compose.

Technologie użyte w projekcie

W projekcie wykorzystano następujące technologie:

- Python 3.14 - język programowania
- Flask – framework webowy (interfejs użytkownika)
- asyncio – asynchroniczne operacje sieciowe
- multiprocessing – przetwarzanie równoległe
- BeautifulSoup – parsowanie HTML
- MongoDB – baza danych NoSQL
- Docker – konteneryzacja aplikacji
- HTML + CSS – interfejs użytkownika

Instalacja i uruchomienie

Uruchomienie przy użyciu Docker'a:

Na początku aby uruchomić projekt, trzeba sklonować repozytorium:

```
git clone https://github.com/rzepeeeek/Web-Scraper-Project.git
cd WEB-SCRAPER-PROJECT
cd WEB-SCRAPER-PROJECT
```

Następnie trzeba uruchomić kontenery za pomocą polecenia:

```
docker-compose up -build
```

Po uruchomieniu aplikacja jest dostępna pod adresem:

<http://localhost:5000/>

<http://127.0.0.1:5000/>

Po zakończeniu pracy zatrzymać projekt kombinacją klawiszy:

CTRL + C

Następnie wykonać polecenie:

```
docker-compose down
```

Uruchomienie wersji lokalnej:

1. Klonowanie repozytorium:

```
git clone https://github.com/rzepeeeek/Web-Scraper-Project.git
cd WEB-SCRAPER-PROJECT
cd WEB-SCRAPER-PROJECT
```

2. Utworzenie środowiska:

```
python -m venv .venv
.venv\Scripts\activate
```

3. Instalacja zależności:

```
pip install -r requirements.txt
```

4. Uruchomienie MongoDB:

```
mongod --dbpath C:\data\db
```

5. Uruchomienie aplikacji:

```
cd engine
python main.py

cd interface
python app.py
```

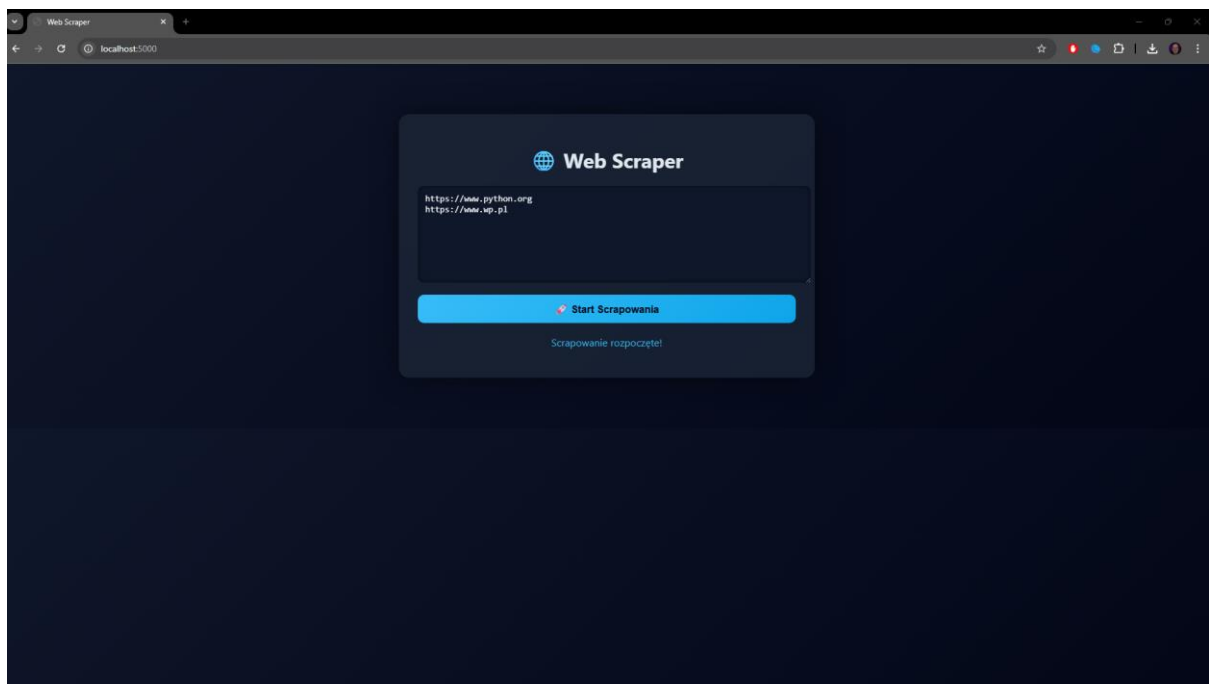
6. Odpalenie aplikacji:

<http://localhost:5000/>

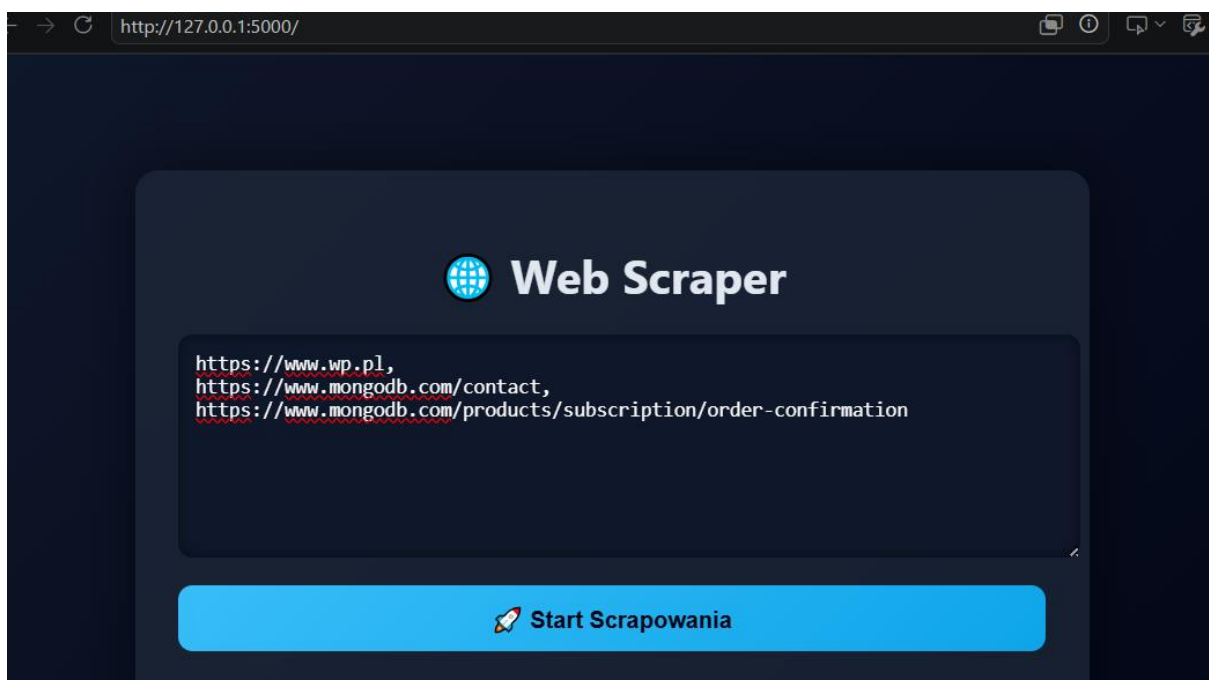
Po odpaleniu strony pojawi się formularz, do którego można wkleić adres URL. Po kliknięciu przycisku “Start Scrapowania” rozpocznie się pobieranie danych, a komunikat “Scrapowanie rozpoczęte” poinformuje o poprawnym rozpoczęciu działania silnika.

Dane będą zapisywane do bazy danych MongoDB w scraper_db -> data.

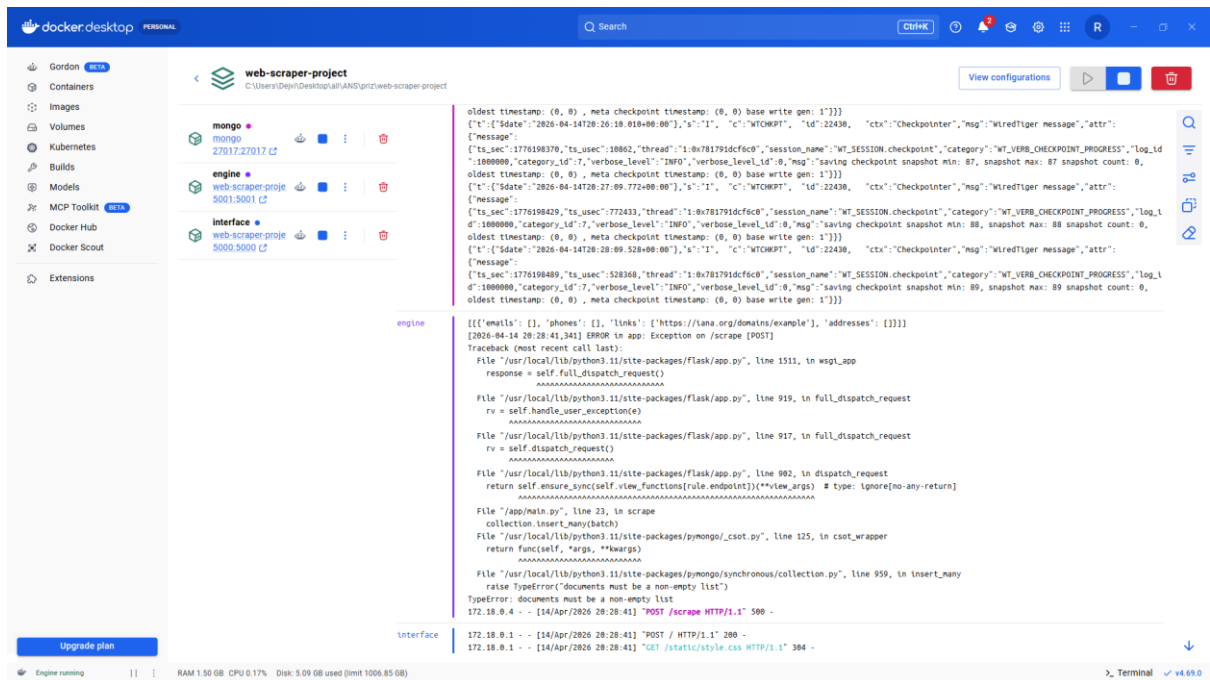
Zdjęcia z aplikacji:



Zdjęcie 1: Główna strona aplikacji.



Zdjęcie 2: Przykładowe strony do scrapowania.



Zdjęcie 3: Docker Desktop i aktualnie uruchomione kontenery projektu.

Przykładowe wyniki scrapowania 3 stron:

localhost:27017 > scraper_db > data

Documents 3

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

UPDATE

DELETE

EXPORT DATA

EXPORT CODE

```
_id: ObjectId('6a271ac70413ca16b7a6ae93')
titles: "Wirtualna Polska - Wszystko co ważne - www.wp.pl"
▶ emails: Array (empty)
▶ phones: Array (800)
▶ links: Array (455)
▶ addresses: Array (empty)
▶ headers: Array (1)
▶ subheaders: Array (7)
description: "Nowoczesne medium, porządkuje świat i dostarcza angażujące informacje,..."
```

```
_id: ObjectId('6a271ac70413ca16b7a6ae94')
titles: "Contact Us | MongoDB"
▶ emails: Array (2)
▶ phones: Array (68)
▶ links: Array (94)
▶ addresses: Array (empty)
▶ headers: Array (1)
▶ subheaders: Array (1)
description: "We are here to help. Get in touch with sales, support or our press tea..."
```

```
_id: ObjectId('6a271ac70413ca16b7a6ae95')
titles: "Order Confirmation | MongoDB"
▶ emails: Array (2)
▶ phones: Array (17)
▶ links: Array (91)
▶ addresses: Array (empty)
▶ headers: Array (1)
▶ subheaders: Array (empty)
```

Zdjęcie 4: Zescapowane przykładowe strony.

```

▶ _id: ObjectId('6a271ac70413ca16b7a6ae93')
  titles: "Wirtualna Polska - Wszystko co ważne - www.wp.pl"
  ▶ emails: Array (empty)
  ▶ phones: Array (800)
  ▶ links: Array (455)
  ▶ addresses: Array (empty)
  ▼ headers: Array (1)
    0: "wp.pl"
  ▼ subheaders: Array (7)
    0: "USŁUGI WP"
    1: "STREFA SPORTU"
    2: "STREFA BIZNESU"
    3: "Oferty"
    4: "Gazetki promocyjne"
    5: "MOTO - TECH - SPRZĘT"
    6: "PORADY KONSUMENCKIE"
  description: "Nowoczesne medium, porządkuje świat i dostarcza angażujące informacje,..."

```

Zdjęcie 5: Zescrapowane wp.pl.

```

_id: ObjectId('6a271ac70413ca16b7a6ae94')
titles: "Contact Us | MongoDB"
▼ emails: Array (2)
  0: "13x13-dot-dark@3x-wmem1vevpx.svg"
  1: "smoothscroll-polyfill@0.4.4"
▼ phones: Array (68)
  0: "2024-09-11"
  1: "2020-04-01"
  2: "2635 28 14"
  3: "5742359816699904"
  4: "1080-4165-9"
  5: "2023-10-20"
  6: "05-18-2026"
  7: "2023-06-30"
  8: "2560011531"

```

Zdjęcie 6: Zescrapowane mongodb.com/contact.


```

8: "8560011531"
9: "2024-09-16"
10: "+54 11 5984 2203"
11: "+55 21 3500 0460"
12: "2026-01-14"
13: "2019-09-17"
14: "1582745430130"
15: "2022-05-18"
16: "2025-11-21"
17: "+44 203 930 0584"
18: "2020-08-11"
19: "2019-11-14"
20: "2023-01-11"
21: "2024-12-23"
22: "2026-04-27"
23: "2020-03-17"
24: "2021-02-04"

```

▼ Show 43 more fields in phones

▢ links : Array (94)

▼ addresses : Array (empty)

▼ headers : Array (1)

0: "Contact Us"

▼ subheaders : Array (1)

0: "Submit an inquiry"

description : "We are here to help. Get in touch with sales, support or our press tea..."

Zdjęcie 7: Ciąg dalszy zescrapowanej strony mongodb.com/contact.

```

_id: ObjectId('6a271ac70413ca16b7a6ae95')
titles : "Order Confirmation | MongoDB"
▼ emails : Array (2)
  0: "info@mongodb.com"
  1: "smoothscroll-polyfill@0.4.4"
▼ phones : Array (17)
  0: "2021-03-16"
  1: "1613160886605"
  2: "2021-02-12"
  3: "2026-05-13"
  4: "2635 28 14"
  5: "1613160867255"
  6: "2023-10-20"
  7: "2021-02-04"
  8: "05-18-2026"
  9: "7362 28 15"
  10: "2021-01-15"
  11: "2025-11-29"
  12: "6969449416"
  13: "2021-06-09"
  14: "2025-11-10"
  15: "2021-10-19"
  16: "15508090763"
▢ links : Array (91)
▢ addresses : Array (empty)
▼ headers : Array (1)
  0: "Order confirmation"
▼ subheaders : Array (empty)
description : "Thanks for your MongoDB Subscription order! This is your confirmation ..."
```

Zdjęcie 8: Zescrapowane mongodb.com/products/subscription/order-confirmation.

Przykładowe sprawdzenie scrapowanych stron za pomocą terminala w VSC:

Wyświetlenie listy aktualnie uruchomionych kontenerów Docker'a oraz uruchomienie mongosh wewnątrz kontenera MongoDB:

```
PS C:\Users\dawid\Documents\web-scraper-project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS         NAMES
82cc89ae1570   web-scraper-project-interface       "python app.py"         3 minutes ago Up 2 minutes
0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   web-scraper-project-interface-1
008bf173057c   web-scraper-project-engine          "python main.py"        3 minutes ago Up 2 minutes
0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp   web-scraper-project-engine-1
800ea34457a1   mongo                               "docker-entrypoint.s..." 3 minutes ago Up 2 minutes
0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp web-scraper-project-mongo-1
PS C:\Users\dawid\Documents\web-scraper-project> docker exec -it web-scraper-project-mongo-1 mongosh
Current Mongosh Log ID: 6a271b75badab060e09df8a2
```

Sprawdzenie zapisanych wyników scrapowania:

```
test> use scraper_db
switched to db scraper_db
scraper_db> |
```

```
scraper_db> db.data.find().pretty()
```

```

_id: ObjectId('6a271ac70413ca16b7a6ae94'),
titles: 'Contact Us | MongoDB',
emails: [ '13x13-dot-dark@3x-wmem1vevpix.svg', 'smoothscroll-polyfill@0.4.4' ],
phones: [
  '2024-09-11',      '2020-04-01',      '2635 28 14',
  '5742359816699904', '1080-4165-9',      '2023-10-20',
  '05-18-2026',      '2023-06-30',      '8560011531',
  '2024-09-16',      '+54 11 5984 2203', '+55 21 3500 0460',
  '2026-01-14',      '2019-09-17',      '1582745430130',
  '2022-05-18',      '2025-11-21',      '+44 203 930 0584',
  '2020-08-11',      '2019-11-14',      '2023-01-11',
  '2024-12-23',      '2026-04-27',      '2020-03-17',
  '2021-02-04',      '2020-08-25',      '2026-01-07',
  '2025-07-24',      '6969449416',      '2021-03-25',
  '2024-03-05',      '2021-10-19',      '+61 1300 103 899',
  '2023-02-27',      '2020-07-22',      '2023-10-27',
  '2026-05-13',      '2026-04-07',      '+52 55 4162 9117',
  '2021-06-09',      '2020-02-26',      '2026-03-06',
  '2025-01-07',      '2020-04-24',      '2020-06-02',
  '2023-10-31',      '+8610 5737 2589', '2025-01-03',
  '2024-10-21',      '+82 10 2672 3710', '2019-11-13',
  '2022-09-22',      '2023-03-07',      '2024-10-22',
  '2019-11-12',      '2020-02-27',      '7362 28 15',
  '2021-01-15',      '2020-10-06',      '2025-02-12',
  '2021-07-13',      '2020-03-30',      '2025-01-02',
  '+1 844-666-4632', '2023-01-04',      '1584405697801',
  '2021-01-14',      '+886 2 7743 0622'
]

```

Sprawdzenie, ile stron zostało zescrapowanych:

```

scraper_db> db.data.countDocuments()
3
scraper_db> █

```

Profil pobieranych danych

Aplikacje podczas analizy stron internetowych wyodrębnia następujące grupy danych:

- Tytuł strony (HTML Title)
- Adresy e-mail
- Numery telefonów
- Linki (href)
- Nagłówki H1
- Nagłówki H2
- Opis strony (Meta Description)

Architektura aplikacji

Aplikacja została podzielona na trzy niezależne moduły:

[INTERFACE] -> [ENGINE] -> [MONGO DB]

- Interface odpowiada za komunikację z użytkownikiem i przyjmowanie adresów URL.
- Engine pobiera strony internetowe, analizuje ich zawartość i wyodrębnia dane.
- MongoDB przechowuje wyniki scrapowania.

Każdy z modułów działa w osobnym kontenerze Docker, dzięki czemu mogą być uruchamiane i rozwijane niezależnie od siebie.

Zalety takiego rozwiązania:

- Łatwiejsze zarządzanie projektem,
- Możliwość rozbudowy poszczególnych modułów,
- Oddzielnie interfejsu od logiki aplikacji i bazy danych.

Testowanie

Testowanie aplikacji obejmowało:

- Sprawdzenie poprawności scrapowania różnych stron
- Testy zapisu danych do MongoDB
- Testy komunikacji między modułami
- Testy działania w środowisku Docker

Przykładowe strony testowe:

- <https://www.wp.pl>
- <https://www.mongodb.com/contact>,
- <https://www.mongodb.com/products/subscription/order-confirmation>

Wyniki:

- Poprawne pobieranie danych
- Poprawny zapis do bazy
- Poprawne wyodrębnianie tytułów stron oraz nagłówków HTML
- Poprawne odczytywanie metadanych stron internetowych
- Stablina praca aplikacji

Wnioski

- Zastosowanie asyncio znacząco przyspiesza pobieranie danych
- multiprocessing umożliwia wykorzystanie wielu rdzeni CPU
- MongoDB dobrze sprawdza się przy danych niestrukturalnych
- Docker upraszcza uruchamianie i wdrażanie systemu

Projekt spełnia wszystkie założenia i działa stabilnie.

Dalszy rozwój

Możliwe kierunki rozwoju:

- Dodanie analizy treści stron
- Wykrywanie danych kontaktowych bardziej zaawansowanymi metodami
- Dodanie panelu administracyjnego
- Obsługa kolejek
- Rozbudowa parsera o wykrywanie dodatkowych typów danych (np. Adresów pocztowych i obrazów)

Dodatkowe funkcjonalności

Możliwość skalowania

System może zostać rozszerzony poprzez:

- Zwiększenie liczby procesów
- Uruchamianie wielu instancji silników
- Wdrożenie w środowisku chmurowym

Modularność i łatwa rozbudowa

Dzięki podziałowi na moduły:

- Można łatwo wymienić bazy danych
- Można dodać nowe typy danych
- Można rozszerzyć frontend bez zmian backendu

PODSUMOWANIE

Projekt stanowi kompletny system do scrapowania danych z Internetu, spełniający wszystkie wymagania zadania.

Aplikacja umożliwia automatyczne wyodrębnianie wielu grup danych ze stron internetowych, takich jak tytuły, adresy e-mail, numery telefonów, odnośniki, nagłówki oraz metadane.

Zastosowane technologie zapewniają wydajność, skalowalność oraz możliwość dalszego rozwoju.